

Test Driven Development (TDD)

Método para desarrollar software
construir código desarrollado por pruebas

NO es una metodología de prueba
lo implementamos con una us.

Requerimientos

Análisis

Diseño

Implementación → 33% al 35%

Prueba / Testing → caso de prueba. 30% al 35% del costo total del proyecto.
dependiendo del tipo de software que sea.

Despliegue → Tiene impacto en la vida de las personas.

Software → actividad humano-intensiva
80% del costo del esfuerzo

Prueba → Actividad programada cronológicamente.
No es necesario tener todo el código programado para empezar con las actividades de prueba.

Caso de prueba → Identificación de situaciones concretas y configuración de datos.
de seteo necesarios para identificar si se cumplen o no criterios específicos.

Se diseña todo lo que necesito para poder probar basandome en los requerimientos
y diseño los casos de prueba.
No se estiman los tiempos de testing ni de desarrollo o corrección.

Se debe pensar el testing desde el principio.
Adelantar el diseño de los casos de prueba.

Muy humano → cometer errores y no asumir la responsabilidad / culpar a otros.

La calidad del producto es responsabilidad de todos, NO de testing.

Despliegue: Producción
Se comienza a usar.
Acciones para modificar el software.

Debo encargarme de que el
código construido funcione
en producción y de manteni-
miento.

DevOps → Rol
Development + Operación

Niveles de prueba → (4)

1. Prueba unitaria (propio desarrollador)
Validar y verificar. Implementar req. funcionales ⊕ req. no funcionales.
no olvidar incluir

Una de las trece prácticas de extreme programming.

2. Integración
Generar un build.
Testing

3. Sistema / Versión.
Equipo de testing.

4. UAT
Prueba de aceptación de usuario } P.O.
User Acceptance Testing } cruzado.

El análisis se encuentra muy ligado al diseño → Se toman como una sola cosa.

BDD → Desarrollo conducido por comportamiento
Colaboración entre desarrolladores.

Siempre se debe adelantar el trabajo de testing lo más que se pueda.

Leyes

1. No comenzar a escribir código hasta no escribir un caso que falle.
2. Con solo un test que falle es suficiente.
3. No escribir más código del necesario para que el test pase.

Test → Cumpla → Falle → Cambio → Pasa → Mejora

Re-factoring

Introducir cambios reduciendo la posibilidad de introducir errores.

Mejora la calidad.

Más probabilidades de que se haga mejor.

Lo suma TDD → Mejora la calidad interna del componente.

Lo utiliza para encontrar un código que "ande mal".

Cada corrección que se hace suma tres errores más. → Hay que volver a probar todo

Taxi - Mobile - Caso práctico

Pseudocódigo de la VS.

Debo tener las funciones que voy a probar.

Cómo mínimo debo cubrir las de la VS.

→ Precondición / Seteo

def test_nombre VS - camino - resultado (sucess / fail) {}

// precondiciones → todo lo que se necesita para poder ejecutar la prueba.

// caso de prueba

a. assert: palabra clave. Se usa para verificar una condición en la prueba. Si es falsa la prueba falla.

// resultados

b. assert all: se tienen que cumplir todas las condiciones para que se cumpla la prueba.

NOTA // mensaje final

Si todos los assert pasaron. Si paro o no el test.